

Relazione Progetto Lab. Applicazioni Mobili

Ultiwear

Realizzata da Alessandro Nanni

Email: alessandro.nanni17@studio.unibo.it

Matricola: 0001027757

Indice

1	Overview dell'app	1
1.1	Note tecniche	1
1.2	Primo avvio dell'app	1
2	Guardaroba/Wardrobe	2
3	Esplora/Browse	3
4	Eventi/Upcoming Events	4
5	Scambi/Trades	5

1 Overview dell'app

L'applicazione Ultiwear è progettata per giocatori di frisbee con la passione per lo scambio di capi d'abbigliamento.

Tramite essa è possibile visualizzare i propri capi in un armadio digitale, pubblicarli, mostrare interesse per eventuali scambi, consultare i tornei futuri e parteciparvi, e gestire scambi in tempo reale.

Dato che l'app non vuole sostituire il momento dello scambio in persona, e lasciare un pò di mistero e magia, non ci sono username pubblici e non si ha modo di comunicare con gli altri.

1.1 Note tecniche

Quasi tutti i dati utilizzati sono salvati in un database *firebase*: questo include sia le rappresentazioni in forma di tabelle di oggetti, che le immagini.

L'app è realizzata con *jetpack compose*: non ci sono file XML contenenti *view* nella cartella *res*.

L'autenticazione è effettuata tramite account Google.

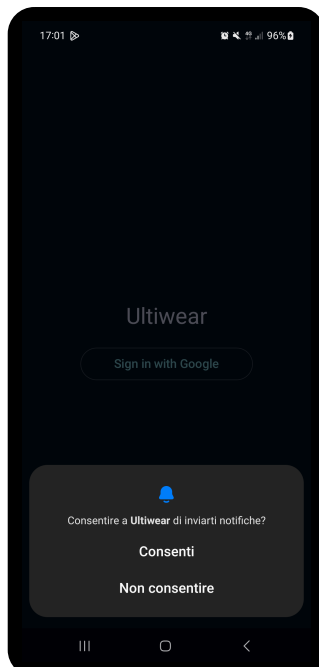
Come consigliato dalla [documentazione ufficiale](#), l'app utilizza una singola attività, la navigazione tra le diverse sezioni è gestita tramite lo stato di Compose.

Si cerca di rispettare il modello MVVM, sono presenti *package* con compiti precisi:

- *data*: comunicazione con il database;
- *model*: contiene le *data class*
- *notifications*: si occupa dell'invio di notifiche;
- *view*: contiene i *composable*;
- *viewModel*: contiene i *ViewModel* intermediari tra *View* e *Model*

L'app utilizza il *MaterialTheme* di Android.

1.2 Primo avvio dell'app



La prima volta che si avvia l'app verranno richiesti i permessi necessari: invio di notifiche, posizione e accesso alla fotocamera, richiesti dal manifest.

```
AndroidManifest.xml
1  <uses-permission
    android:name="android.permission.CAMERA" />
2  <uses-permission
    android:name="android.permission.POST_NOTIFICATIONS"/
    >
3  <uses-permission
    android:name="android.permission.ACCESS_FINE_LOCATION
    >
```

Nella *MainActivity*, questi vengono richiesti in catena affinché l'ultimo (*locationPermission*), non sovrascriva i precedenti.

```

1 private fun requestPermissionsOnStartup() {
2     notificationPermissionLauncher.launch(POST_NOTIFICATIONS)
3 }
4 private val notificationPermissionLauncher =
5     registerForActivityResult(ActivityResultContracts.RequestPermission()) { _ ->
6         requestCameraPermission() // next
7     }

```

Listing 1: Dopo aver richiesto il permesso per le notifiche, si chiederà quello per la fotocamera

Successivamente, tramite stati *composable* dell' `authViewModel`, si controlla che l'utente sia registrato e connesso a internet.



In base a questi stati si mostreranno 3 *view* diverse. Dopo aver eseguito l'accesso con Google, l'utente potrà utilizzare l'app.

```

1 GetCredentialRequest.Builder().addCredentialOption(
2     GetGoogleIdOption.Builder()
3         .setFilterByAuthorizedAccounts(false)
4         .setServerClientId(BuildConfig.GOOGLE_CLIENT_ID)
5         .setAutoSelectEnabled(false)
6         .build()).build()

```

Listing 2: Codice per creare il prompt “sign in with Google” e ottenere un token di autenticazione

Il token di login sarà poi convertito in un token Firebase, memorizzato in una collezione come UID assieme alla mail dell'utente. Ora l'utente ha accesso all'app, e potrà navigare le sue 5 *view*.

`var selectedIndex by rememberSaveable { mutableIntStateOf(0) }` tiene traccia della *sub-activity* corrente. Ognuna di esse è definita dal *composable* `TabItem`.

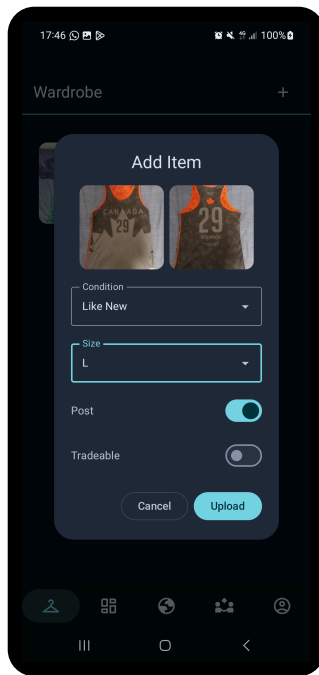
```

1 val tabs = listOf(
2     TabItem(
3         "Wardrobe", R.drawable.wardrobe
4     ) {
5         WardrobeScreen(wardrobeViewModel)
6     },
7     ...
8 )

```

2 Guardaroba/Wardrobe

Qui l'utente può inserire dati e foto relativi ai suoi capi d'abbigliamento.



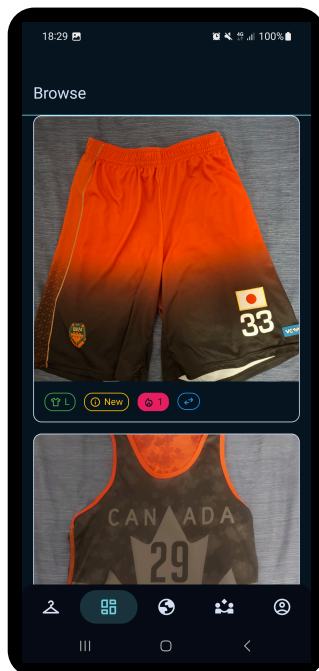
Per ogni capo si può fare una foto del fronte, retro, specificare taglia e condizioni, e infine scegliere se pubblicare¹ e indicare se questo capo lo si vuole scambiare. I capi sono mostrati tramite una `LazyVerticalGrid` di 3 colonne. Cliccando un elemento della griglia si potranno visualizzare i dettagli del capo.

Le operazioni sul database sono effettuate tramite l'oggetto `ItemHandler`, di cui il `WardrobeViewModel` possiede un riferimento. Un listener controlla continuamente cambiamenti nella collezione `wardrobe` per gli oggetti dell'owner. In caso di cambiamenti si aggiornano 3 liste di item: una che li contiene tutti, una che contiene solo quelli scambiabili e un'ultima che contiene solo quelli posseduti (`items.filter { it.owned }`). Il listener permette di aggiornare automaticamente gli *item* dell'utente in ogni punto dell'app (*Wardrobe*, *Browse* e *Trade*).

Ogni oggetto capo contiene la variabile booleana `owned`. Questa è usata per indicare se il capo deve essere mostrato nell'armadio, nel caso l'utente corrente non ne sia più il proprietario in seguito ad uno scambio².

Inizialmente per gestire la cancellazione di un capo si eliminava il documento corrispondente. Dopo aver aggiunto i post e timeline degli scambi, è stato utilizzato un metodo che non elimina i propri capi, ma li nasconde solamente. Il post associato invece viene eliminato, ma dato che l'item originale esiste ancora sul database, vi si può fare riferimento per mostrare gli item dati via in uno scambio.

3 Esplora/Browse



In questa sezione si possono vedere le foto e dettagli degli *item* postati. Per ogni post non proprio si può anche mettere like e se il proprietario lo ha permesso, esprimere interesse di scambio. I bottoni di scambio e like sono oscurati e non cliccabili per i propri capi d'abbigliamento. Ogni post è contenuto in una `LazyColumn`, ed è una `Card` composta da un `Pager` che si può scorrere per vedere le foto di fronte e retro e una `InfoBar`.

Il `BrowseViewModel` contiene un riferimento allo stesso handler, e dunque listener del di *Wardrobe*. Dunque, quando un utente preme il bottone like, il `ViewModel` viene notificato di questo cambiamento e ricarica la colonna. Per evitare che un utente possa mettere like allo stesso post più volte, ogni post dispone di una collezione che contiene gli id degli utenti che hanno messo like.

Similmente, la collezione `trade_interests` tiene traccia degli utenti che hanno espresso interesse a scambiare un certo item.

```
1 try {
2     val newLikes = handler.toggleLike(wardrobeUid, currentUser.uid)
3     // update state
```

kt

¹Visibile nella sezione *Browse*, mostrata in seguito.

²Eseguibile nella sezione *Quick Trade*, mostrata in seguito.

```

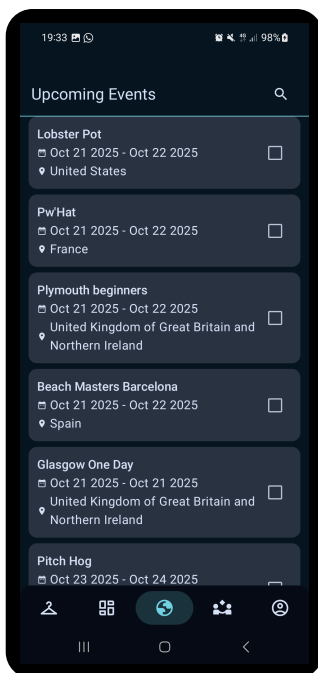
4  _items.value = _items.value.map { item ->
5      if (item.wardrobeUid == wardrobeUid) {
6          item.copy(post = item.post?.copy(likes = newLikes))
7      } else item
8  }
9  } catch (e: Exception) {
10     Log.e(tag, "Error toggling like", e)
11 }

```

I like possono essere messi e tolti: il handler si occupa di aggiornare il database con una transazione per rendere atomica l'operazione di lettura e scrittura. Il ViewModel crea una copia degli *items*, e se l'id di uno di essi corrisponde con quello a cui è stato cambiato il like, si aggiorna il valore mostrato nella *card*.

4 Eventi/Upcoming Events

Qui si possono vedere i prossimi tornei da tutto il mondo, ottenuti tramite l'api di [Ultical](#). Sono *card* ordinate dalla data di inizio più vicina alla più lontana. Ogni card ha associata una *checkbox* per indicare se si sarà presenti ad un certo torneo.



Il singleton `APIClient` inizializza una sola volta (tramite `by lazy`) l'interfaccia `UlticalAPI` fornendogli l'url a cui fare la chiamata HTTP, e una factory da usare per processare i dati. In questo caso si usa la libreria GSON di Google in quanto l'API di `ultical` restituisce un oggetto JSON. Retrofit è la libreria usata per fare chiamate HTTP.

```

1  val api: UlticalApi by lazy {
2      Retrofit.Builder()
3          .baseUrl(URL)
4          .addConverterFactory(
5              GsonConverterFactory.create()
6          )
7          .build()
8          .create(UlticalApi::class.java)
9  }

```

L'interfaccia fornisce un metodo `getEvents()` per ottenere una lista di eventi.

Quando viene aperta questa sezione per la prima volta, prima si esegue un “flattening” dell'oggetto JSON: dato che un torneo ha una lista di edizioni, ciascuna con un ID, si esegue questa operazione per lavorare su dati più omogenei. Nella lista appiattita vengono inclusi solamente gli eventi con data di inizio maggiore o uguale a quella attuale.

Quando un utente si dichiara presente ad un torneo, non solo si aggiorna la collezione su Firebase, ma un *listener* sulla collezione `user_attendances` modifica la `MutableMap` `attendances`, che per ogni ID, associa un booleano per indicare se l'utente sarà presente ad un certo torneo. Questo esempio evidenzia perfettamente l'utilità del ViewModel: qualora ci fosse bisogno di ricomporre lo schermo, non è necessario fare una nuova chiamata all'API, dal momento che i valori sono già memorizzati `EventViewModel`.

Quando si apre nuovamente la schermata eventi dopo aver chiuso l'app, le presenze sono ripristinate

sulla *view* perché ogni *card* dispone di un riferimento al `ViewModel`, che può usare per controllare se l'utente è presente al torneo con id pari a quello della *card* che si sta componendo.

5 Scambi/Trades

In questa *view* si possono vedere e fare tutte le operazioni relative agli scambi, interagendo direttamente o indirettamente con altri utenti.

La sezione degli scambi è costituita da 3 *view*. Quella attualmente in vista è controllata da una state variable di *compose*

```
1 var selectedTab by remember { mutableIntStateOf(0) }
2 val tabs = listOf("Matches", "Manual Trade", "Quick Trade")
```

kt

Per ogni tab, se la sua posizione nella lista coincide con l'indice, il suo sfondo viene colorato per evidenziare che è quella selezionata. Un altro stato, `showTradesDialog` (booleano) indica se bisogna mostrare il dialogo che contiene lo storico degli scambi.